

CPU Instruction Set

If you are looking for textual explanations of what each instruction does, please read [gbz80\(7\)](#); if you want a compact reference card/cheat sheet of each opcode and its flag effects, please consult [the optables](#) (whose [octal view](#) makes most encoding patterns more apparent).

The Game Boy's SM83 processor possesses a CISC, variable-length instruction set. This page attempts to shed some light on how the CPU decodes the raw bytes fed into it into instructions.

The first byte of each instruction is typically called the "opcode" (for "operation code"). By noticing that some instructions perform identical operations but with different parameters, they can be grouped together; for example, `inc bc`, `inc de`, `inc hl`, and `inc sp` differ only in what 16-bit register they modify.

In each table, one line represents one such grouping. Since many groupings have some variation, the variation has to be encoded in the instruction; for example, the above four instructions will be collectively referred to as `inc r16`. Here are the possible placeholders and their values:

	0	1	2	3	4	5	6	7
r8	b	c	d	e	h	l	[hl]	a
r16	bc	de	hl	sp				
r16stk	bc	de	hl	af				
r16mem	bc	de	hl+	hl-				
cond	nz	z	nc	c				
b3	A 3-bit bit index							
tgt3	rst 's target address, divided by 8							
imm8	The following byte							
imm16	The following two bytes, in little-endian order							

These last two are a little special: if they are present in the instruction's mnemonic, it means that the instruction is 1 (`imm8`) / 2 (`imm16`) extra bytes long.

[hl+] and [hl-] can also be notated [hli] and [hld] respectively (as in increment and decrement).

Groupings have been loosely associated based on what they do into separate tables;

those have no particular ordering, and are purely for readability and convenience. Finally, the instruction “families” have been further grouped into four “blocks”, differentiated by the first two bits of the opcode.

Block 0

	7	6	5	4	3	2	1	0
nop	0	0	0	0	0	0	0	0

	7	6	5	4	3	2	1	0
ld r16, imm16	0	0	Dest (r16)		0	0	0	1
ld [r16mem], a	0	0	Dest (r16mem)		0	0	1	0
ld a, [r16mem]	0	0	Source (r16mem)		1	0	1	0
ld [imm16], sp	0	0	0	0	1	0	0	0

	7	6	5	4	3	2	1	0
inc r16	0	0	Operand (r16)		0	0	1	1
dec r16	0	0	Operand (r16)		1	0	1	1
add hl, r16	0	0	Operand (r16)		1	0	0	1

	7	6	5	4	3	2	1	0
inc r8	0	0	Operand (r8)		1	0	0	
dec r8	0	0	Operand (r8)		1	0	1	

	7	6	5	4	3	2	1	0
ld r8, imm8	0	0	Dest (r8)		1	1	0	

	7	6	5	4	3	2	1	0
rlca	0	0	0	0	0	1	1	1
rrca	0	0	0	0	1	1	1	1
rla	0	0	0	1	0	1	1	1
rra	0	0	0	1	1	1	1	1
daa	0	0	1	0	0	1	1	1
cpl	0	0	1	0	1	1	1	1
scf	0	0	1	1	0	1	1	1
ccf	0	0	1	1	1	1	1	1

	7	6	5	4	3	2	1	0
jr imm8	0	0	0	1	1	0	0	0
jr cond, imm8	0	0	1	Condition (cond)		0	0	0

	7	6	5	4	3	2	1	0
stop	0	0	0	1	0	0	0	0

stop is often considered a **two-byte** instruction, though [the second byte is not always ignored](#).

Block 1: 8-bit register-to-register loads

	7	6	5	4	3	2	1	0
ld r8, r8	0	1	Dest (r8)			Source (r8)		

Exception: trying to encode `ld [hl], [hl]` instead yields [the halt instruction](#):

	7	6	5	4	3	2	1	0
halt	0	1	1	1	0	1	1	0

Block 2: 8-bit arithmetic

	7	6	5	4	3	2	1	0
add a, r8	1	0	0	0	0	Operand (r8)		
adc a, r8	1	0	0	0	1	Operand (r8)		
sub a, r8	1	0	0	1	0	Operand (r8)		
sbc a, r8	1	0	0	1	1	Operand (r8)		
and a, r8	1	0	1	0	0	Operand (r8)		
xor a, r8	1	0	1	0	1	Operand (r8)		
or a, r8	1	0	1	1	0	Operand (r8)		
cp a, r8	1	0	1	1	1	Operand (r8)		

Block 3

	7	6	5	4	3	2	1	0
add a, imm8	1	1	0	0	0	1	1	0
adc a, imm8	1	1	0	0	1	1	1	0
sub a, imm8	1	1	0	1	0	1	1	0
sbc a, imm8	1	1	0	1	1	1	1	0
and a, imm8	1	1	1	0	0	1	1	0
xor a, imm8	1	1	1	0	1	1	1	0

	7	6	5	4	3	2	1	0
or a, imm8	1	1	1	1	0	1	1	0
cp a, imm8	1	1	1	1	1	1	1	0

	7	6	5	4	3	2	1	0
ret cond	1	1	0	Condition (cond)		0	0	0
ret	1	1	0	0	1	0	0	1
reti	1	1	0	1	1	0	0	1
jp cond, imm16	1	1	0	Condition (cond)		0	1	0
jp imm16	1	1	0	0	0	0	1	1
jp hl	1	1	1	0	1	0	0	1
call cond, imm16	1	1	0	Condition (cond)		1	0	0
call imm16	1	1	0	0	1	1	0	1
rst tgt3	1	1	Target (tgt3)			1	1	1

	7	6	5	4	3	2	1	0
pop r16stk	1	1	Register (r16stk)		0	0	0	1
push r16stk	1	1	Register (r16stk)		0	1	0	1

	7	6	5	4	3	2	1	0
Prefix (see block below)	1	1	0	0	1	0	1	1

	7	6	5	4	3	2	1	0
ldh [c], a	1	1	1	0	0	0	1	0
ldh [imm8], a	1	1	1	0	0	0	0	0
ld [imm16], a	1	1	1	0	1	0	1	0
ldh a, [c]	1	1	1	1	0	0	1	0
ldh a, [imm8]	1	1	1	1	0	0	0	0
ld a, [imm16]	1	1	1	1	1	0	1	0

	7	6	5	4	3	2	1	0
add sp, imm8	1	1	1	0	1	0	0	0
ld hl, sp + imm8	1	1	1	1	1	0	0	0
ld sp, hl	1	1	1	1	1	0	0	1

	7	6	5	4	3	2	1	0
di	1	1	1	1	0	0	1	1
ei	1	1	1	1	1	0	1	1

The following opcodes are **invalid**, and **hard-lock the CPU** until the console is powered off: \$D3, \$DB, \$DD, \$E3, \$E4, \$EB, \$EC, \$ED, \$F4, \$FC, and \$FD.

\$CB prefix instructions

	7	6	5	4	3	2	1	0
rlc r8	0	0	0	0	0	Operand (r8)		
rrc r8	0	0	0	0	1	Operand (r8)		
rl r8	0	0	0	1	0	Operand (r8)		
rr r8	0	0	0	1	1	Operand (r8)		
sla r8	0	0	1	0	0	Operand (r8)		
sra r8	0	0	1	0	1	Operand (r8)		
swap r8	0	0	1	1	0	Operand (r8)		
srl r8	0	0	1	1	1	Operand (r8)		

	7	6	5	4	3	2	1	0
bit b3, r8	0	1	Bit index (b3)			Operand (r8)		
res b3, r8	1	0	Bit index (b3)			Operand (r8)		
set b3, r8	1	1	Bit index (b3)			Operand (r8)		